

RESEARCH

Open Access



Deep learning enables precision agriculture for sustainable chili pepper disease detection in Benin

Mireille Gloria Founmilayo Odounfa^{1*}, Castro Gbêmémali Hounmenou^{1,2,3}, Valère Kolawolé Salako¹, Antoine Affokpon⁴ and Romain L. Glèlè Kakai¹

*Correspondence:

Mireille Gloria Founmilayo Odounfa
odounfemi1@yahoo.com

¹Laboratoire de Biomathématiques
et d'Estimations Forestières (LABEF),
Faculty of Agronomic Sciences,
University of Abomey-Calavi, 04, BP
1525, Cotonou, Benin

²Département Informatique,
Université de Labé, Labé, Guinea

³Centre de Recherche et de
Formation en Infectiologie de
Guinée, Conakry, Guinea

⁴Laboratoire d'Etude et de Gestion
des Organismes Nuisibles des
Végétaux (LaGON), Faculty of
Agronomic Sciences, University
of Abomey-Calavi, 04, BP 1525,
Cotonou, Benin

Abstract

Ensuring food security is a crucial priority for nations worldwide, but plant diseases significantly hinder this objective, through their impacts on agricultural productivity. Chili pepper (*Capsicum spp.*) is a major crop in West Africa including in Benin. However, its production is challenged by diseases such as anthracnose and Tomato Yellow Leaf Curl Virus (TYLCV) which severely impact its yields and farmer livelihoods. While traditional methods for disease detection and management have been commonly used, they are no longer sufficient to combat rising pest infestations and declining agricultural productivity. To tackle this issue, we built a comprehensive dataset of 213 images of anthracnose-affected leaves, 202 images of TYLCV-infected leaves, and 119 images of healthy leaves, collected under diverse environmental conditions in Benin. The study compared the performance of thirteen (13) deep learning models including among others YOLOv8, MobileNetV2, and DenseNet121 for the classification of chili diseases in Benin using transfer learning techniques to improve their accuracy, based on values close to one (1) of evaluation metrics such as Accuracy, Precision, Recall and F1-Score. Results show that YOLOv8 outperforms other models in detecting and localizing leaf diseases in real time. It achieved a mean Average Precision (mAP@0.5) of 0.995 and mAP@0.5–0.95 of 0.941, demonstrating strong generalization. With precision and recall exceeding 99%, YOLOv8 proves highly reliable for agricultural applications. Among classification models, MobileNetV2 and DenseNet121 achieved 96.25% accuracy, confirming their effectiveness in disease classification. Our findings demonstrate that deep learning models, particularly YOLOv8, hold immense potential for real-time, automated detection of chili pepper diseases. Future research should focus on expanding datasets, integrating climatic variations, and improving disease severity assessment for enhanced agricultural sustainability.

Keywords Deep learning, Real-time detection, Pepper diseases, Capsicum, Agricultural sustainability



1 Introduction

Chili peppers (*Capsicum spp.*) are a crucial staple crop in Africa and particularly in Benin, playing a vital role in culinary traditions, local economies, and food security [18]. As the second most important vegetable crop after tomatoes in Benin, chili peppers are extensively cultivated by smallholder farmers across the country, contributing to the livelihoods of countless rural communities [48, 58]. The annual crop production at the national level was estimated at 1,76,535 tons in 2016 [17]. The pungent, flavorful fruits are not only indispensable ingredients in Beninese cuisine but also generate significant income through both domestic and export markets [48]. Despite the economic and cultural significance of chili peppers in Benin, the cultivation of this crop faces numerous challenges, primarily due to the prevalence of devastating plant diseases. Two of the most troublesome diseases affecting chili peppers in the region are anthracnose, caused by the fungal pathogen *Colletotrichum spp.*, and Tomato Yellow Leaf Curl Virus (TYLCV), transmitted by whitefly vectors (*Bemisia tabaci*.) [18, 21, 48, 69]. These diseases can lead to substantial yield losses, often exceeding 50%, thereby threatening the livelihoods of chili pepper farmers and the food security of local communities [19]. Traditionally, Beninese farmers and agricultural extension workers have relied on visual inspection and manual sampling methods to identify disease symptoms in chili pepper plants [60]. However, this approach is labor-intensive, time-consuming, and heavily dependent on individual expertise, thus, frequently resulting in inaccurate diagnoses. Furthermore, the similar symptomology of various diseases can confound the accurate identification of the causal agents, hindering the implementation of targeted and effective interventions [18, 72]. These challenges pledge for an urgent need to implement an autonomous and automatic biometric system to disease diagnosis to ultimately improve chili production.

Precision agriculture is emerging as a promising solution, leveraging cutting-edge technologies and data-driven methodologies to increase agricultural productivity [9, 12]. With advancements in computer vision and graphics processing units, disease detection has primarily been categorized into two main approaches: traditional machine vision techniques and deep learning-based target detection methods [39]. Several tasks including classification tasks are carried out by different techniques. However, a series of steps must be followed to achieve classification using conventional machine learning techniques, including pre-processing, feature extraction, optimal feature selection, learning, and classification. It is important to note that feature selection plays a significant role in determining the performance of these machine-learning techniques. Biased feature selection can result in incorrect discrimination among classes. On the other hand, deep learning (DL), a subset of machine learning has the advantage of automating the learning of feature sets for various tasks, unlike conventional ML methods. This means that DL can automatically extract relevant features without the need for explicit feature selection [4, 33]. With its many data technologies and high-performance computing capability, deep learning has become an indispensable technique widely exploited in the agricultural industry [59]. These models enable the analysis of huge image datasets [5]. Convolutional Neural Networks (CNN) are specially DL techniques designed to extract information from images, making them dynamic for disease detection. Deep learning-based systems can operate in real-time in the field, providing instant information to farmers. Automation makes it possible to continuously monitor large areas of crops [42].

Unlike visual methods, deep learning reduces reliance on human expertise, making disease detection and estimation more reliable and reproducible [38]. Deep learning models can also be trained to detect various diseases and adapted to different crops, making them very versatile [68]. These models can detect signs of disease long before they are visible to the naked eye. This allows for early intervention and more effective disease management [2]. The emergence of deep learning has fundamentally transformed approaches to detecting and estimating plant diseases in agriculture [2].

Recent studies have explored the use of deep learning techniques, such as convolutional neural networks (CNNs), for the detection and classification of diseases affecting chili peppers and other related vegetable crops [8]. Naresh et al. [46] introduce an enhanced squeeze and excitation densely connected convolutional neural network (SEDCNN) architecture for the early detection of chili leaf diseases. This model achieves a remarkable disease identification accuracy of 97%, which can be further increased to 98.86% through data augmentation. Pratap et al. [51] presents a high-precision multi-class classification of chili leaf diseases using a customized EfficientNetB4 deep learning model, which outperformed other popular CNN architectures like ResNet-50, DenseNet-121, MobileNet-V2, and VGG-16 in detecting chili leaf diseases. Hang et al. [23] compares the performance of several widely used CNN architectures AlexNet, VGGNet, GoogLeNet, ResNet, and SENet on a plant leaf disease dataset. It proposes an improved CNN architecture that integrates the Inception module, Squeeze-and-Excitation module, and global average pooling to mitigate issues related to long training times and overfitting. This enhanced architecture achieves the highest classification accuracy of 91.7% on the dataset and demonstrates superior robustness compared to other CNN methods. Ahmad et al. [3] demonstrated the effectiveness of YOLOv8 in identifying tomato diseases under variable lighting conditions, while Kumar et al. [37] evaluated its deployment in mobile applications for cassava leaf disease detection. Other research has investigated the use of hybrid models combining CNNs and attention mechanisms to improve feature extraction in complex backgrounds [70]. Despite the promising results of traditional classifiers in previous studies, their effectiveness in real-time field conditions, especially in West African climates, remains underexplored. This study bridges this gap by assessing the feasibility of integrating other object detection models, such as YOLOv8 for precision agriculture in Benin. Naik et al. [45] compares classification and object detection techniques to recognize leaf diseases, highlighting that object detection also provides the location of the disease on the leaf. Using a dataset of 5100 chili leaf images spanning six different classes, the YOLOv5s object detection model was applied and evaluated with performance metrics [45]. The authors propose an enhanced single-stage object detection model that outperformed the YOLOv5s model in terms of performance metrics. Kanaparthi et al. [32] used deep learning methods, specifically YOLOv5 and YOLOv7, to detect chilli pests. The researchers created a new dataset of 13,414 chili pest images across three classes (Black Thrips, Redmites, and White Fly) to improve classification accuracy. The study found that the YOLOv5 algorithm outperformed YOLOv7, VGG-16, and VGG-19 in detecting chili pests, achieving a mean average precision of 98.6%. However, the literature on the application of deep learning for the detection and management of diseases specifically affecting chili peppers cultivated in Benin remains limited.

This study aims to address these research gaps by developing and evaluating deep learning-based models for the early and accurate detection of anthracnose and TYLCV in chili pepper crops cultivated in Benin. Although previous research has explored the use of CNNs or object detection algorithms independently, our work is distinctive in several key aspects. The key contributions and innovations of this work can be summarized as follows: i) A comprehensive dataset of chili pepper leaf images, captured in complex backgrounds under various lighting conditions and angles, has been collected, thus complementing existing datasets. These images, including confirmed cases of anthracnose and TYLCV as well as healthy leaves, come from different agricultural regions of Benin. ii) The study employs state-of-the-art deep learning models, including VGG16, VGG19, ResNet50, MobileNetV2, DenseNet121, InceptionV3, Xception, EfficientNetB0, EfficientNetB1, EfficientNetB2, EfficientNetB3, NASNetMobile, and YOLOv8, leveraging transfer learning to enhance model performance and adaptability. A detailed comparison of various CNN architectures was conducted to determine their effectiveness in detecting and classifying chili pepper diseases. This comparison helps identify the most suitable models for this specific application. The integration of YOLOv8 for simultaneous localization and classification of leaf diseases offers a more practical and deployable solution compared to only classification-based approaches. Our work places a strong emphasis on practical implementation, intending to support smallholder farmers with AI-driven, accessible, and scalable diagnostic tools.

The rest of this paper is structured as follows: Section 2 (Deep Learning Algorithms Explored) introduces the deep learning models compared in this study. Section 3 (Methods) details the processes for collecting and preparing the chili pepper leaf dataset, including image acquisition under various conditions, hardware and software configurations, training parameters, hyperparameter optimization strategies, and validation and testing procedures, along with the evaluation metrics used to measure model performance. Section 4 (Results) presents the study's findings, providing a comprehensive analysis of the performance of different architectures. Section 5 (Discussion) explores the implications of the findings for improving disease monitoring and management in chili pepper crops in Benin, addressing practical challenges faced by farmers. Finally, Section 6 (Conclusion and Future Work) summarizes the study's main contributions and innovations, highlighting their significance for advancing precision agriculture in Benin, and discusses the potential for integrating the proposed deep learning models into real-world agricultural practices to enhance the sustainability and productivity of chili pepper cultivation. It also outlines future research directions, such as developing more sophisticated models and exploring their applicability to other crops and agricultural contexts.

2 Deep learning algorithms explored

Several state-of-the-art CNN architectures were employed to process and analyze image data, enabling real-time detection and disease monitoring in a more efficient and scalable manner. These CNN models include VGG16, VGG19, ResNet50, MobileNetV2, DenseNet121, InceptionV3, Xception, EfficientNet (B0, B1, B2, B3), NASNetMobile, and YOLOv8, each selected for their strengths in image classification and object detection tasks. Comparative analyses have shown that YOLOv8 outperforms earlier YOLO

versions and other object detection algorithms, including Faster R-CNN, in both accuracy and processing speed [16, 57].

These architectures vary in terms of complexity, computational efficiency, and performance. Transfer learning was applied across models, utilizing pre-trained weights from the ImageNet dataset to enhance their ability to recognize disease patterns in chili pepper leaves. The study employed fine-tuning techniques to tailor each model to the specific characteristics of chili pepper disease detection. Table 1 providing a detailed description of deep learning models, their advantages, disadvantages, as well as the hyperparameters and tested values for each algorithm, offering a comparative overview of the configurations used in this study.

3 Methods

3.1 Study site and data collection

The dataset was assembled using a Techno 20 camera (resolution photos: 3840×2160) and an iPhone 15 Pro Max (resolution photos: 24MP and 48MP), capturing a total of 534 images of chili pepper leaves from various localities across Benin, specifically within the Oueme, Atlantique, and Mono regions. The dataset comprises a total of 415 images of sick leaves (including 213 images affected by anthracnose and 202 images showing symptoms of the Tomato Yellow Leaf Curl Virus (TYLCV)) and 119 images of healthy leaves, representing the plant's different states of health (healthy, diseased, and severely affected).

To ensure a comprehensive and representative dataset, images were collected under diverse conditions, including different lighting, angles, and backgrounds. Data collection occurred between December 2023 and June 2024, spanning three distinct periods of the day: morning (7:00–11:00), lunchtime (12:00–14:00), and afternoon (15:00–18:00).

The dataset was developed in collaboration with a phytopathologist, whose expertise ensured accurate identification and classification of plant health states, as well as precise documentation of the symptoms of each disease.

3.2 Data preprocessing

For the YOLOv8n model, the image dataset was first annotated using Labelme, producing JSON files containing precise bounding boxes and class labels. For each image, rectangular bounding boxes were manually drawn to tightly enclose the visible symptoms of the disease. This strategy was deliberately chosen to train the model to localize the specific visual patterns of the disease itself, thereby maximizing the signal-to-noise ratio for the object detector, rather than simply classifying the entire leaf. The ImageDataGenerator and flow_from_dataframe functions from the TensorFlow/Keras library were employed to preprocess and prepare the image data for training. For the majority of models, images are resized to a standard input size of 224×224 pixels, ensuring uniformity across the dataset. However, certain architectures necessitate tailored adjustments: for instance, the NASNetLarge model performs optimally with images resized to 331×331 pixels, while the YOLOv8 model is designed to accommodate larger inputs of 640×640 pixels. To further enhance model performance, pixel values are normalized to a range of 0–1, a critical preprocessing step that standardizes the data and accelerates convergence during training. Beyond preprocessing, data augmentation techniques are applied to artificially diversify the dataset. These techniques include random rotations,

Table 1 Deep learning model descriptions and configurations

Algorithms	Description	Advantages	Disadvantages	Hyperparameters and possible values	Hyperparameter values tested
VGG16	A 16-layer CNN with 13 convolutional layers and 3 fully connected layers, using small 3×3 filters for progressive feature extraction [62, 65]	Strong feature extraction due to stacked 3×3 convolutions: $y^{(l)} = f(W^{(l)} * x^{(l-1)} + b^{(l)})$, where $y^{(l)}$ is the output of layer l ; $W^{(l)}$ is the weight matrix; $*$ denotes convolution; $x^{(l-1)}$ is the input from the previous layer; $b^{(l)}$ is the bias; and $f(\cdot)$ is the activation function [65]. Effective for fine-grained classification	High computational cost, large number of parameters [62]	Optimizer (SGD, Adam, RMSprop, Adagrad, AdamW), Learning Rate ($\eta \in [10^{-6}, 10^{-1}]$), Batch Size (16, 32, 64, 128, 256), Epochs(10, 20, 30, 50, 100, 200, 300, 500, 1000), Loss Function (Softmax Loss, Log Loss, Sparse Categorical Cross-Entropy)	Optimizer: Adam. Learning Rate:0.0001, 0.001, 0.01. Batch Size: 32, Epochs: 50, 100. Loss Function: Softmax Loss
VGG19	A deeper version of VGG16 with 19 layers, improving feature extraction [55]. Uses small 3×3 convolutions with stride and padding techniques	Enhanced spatial control via stride and padding	Increased memory usage, slower inference [55]	Optimizer (SGD, Adam, RMSprop, Adagrad, AdamW), Learning Rate ($\eta \in [10^{-6}, 10^{-1}]$), Batch Size (16, 32, 64, 128, 256), Epochs(10, 20, 30, 50, 100, 200, 300, 500, 1000), Loss Function (Softmax Loss, Log Loss, Sparse Categorical Cross-Entropy)	Optimizer: Adam. Learning Rate:0.0001, 0.001, 0.01. Batch Size: 32, Epochs: 50, 100. Loss Function: Softmax Loss
ResNet50	A 50-layer deep CNN with residual connections to allow deeper networks to train effectively without vanishing gradients [7]	Residual learning with skip connections mitigates gradient vanishing: $y^{(l)} = f(x^{(l)}) + x^{(l)}$, where $y^{(l)}$ is the output of layer l , allowing direct paths for gradients during backpropagation [7]. Allows very deep architectures	Requires more training time, can overfit on small datasets [7]	Optimizer (SGD, Adam, RMSprop, Adagrad, AdamW), Learning Rate ($\eta \in [10^{-6}, 10^{-1}]$), Batch Size (16, 32, 64, 128, 256), Epochs(10, 20, 30, 50, 100, 200, 300, 500, 1000), Loss Function (Softmax Loss, Log Loss, Sparse Categorical Cross-Entropy)	Optimizer: Adam. Learning Rate:0.0001, 0.001, 0.01. Batch Size: 32, Epochs: 50, 100. Loss Function: Softmax Loss
MobileNetV2	A lightweight CNN using depthwise separable convolutions to reduce computation, making it suitable for mobile deployment [29]	Computational efficiency through depthwise separable convolutions: $y = (W_D * x) * W_P$, where W_D represents depthwise filters applied to each input channel separately; and W_P represents pointwise filters that combine features across channels [29]. Suitable for edge devices	Slightly lower accuracy on complex datasets [29]	Optimizer (SGD, Adam, RMSprop, Adagrad, AdamW), Learning Rate ($\eta \in [10^{-6}, 10^{-1}]$), Batch Size (16, 32, 64, 128, 256), Epochs(10, 20, 30, 50, 100, 200, 300, 500, 1000), Loss Function (Categorical Cross-Entropy, Sparse Categorical Cross-Entropy, Binary Cross-Entropy)	Optimizer: Adam. Learning Rate:0.0001, 0.001, 0.01. Batch Size: 32, Epochs: 50, 100. Loss Function: Categorical Cross-Entropy

formula: $O = \left(\frac{W-F+2P}{S} \right) + 1$, where O is the output size; W is the input size; F is the filter size; P is padding; and S is stride [62]. Better feature refinement than VGG16

Table 1 (continued)

Algorithms	Description	Advantages	Disadvantages	Hyperparameters and possible values	Hyperparameter values tested
DenseNet121	Uses dense connections where each layer receives input from all preceding layers, improving gradient flow and reducing parameters [26]	Dense connections enhance feature reuse: $x^{(l)} = H_l([x^{(0)}, x^{(1)}, \dots, x^{(l-1)}])$, where $H_l(\cdot)$ denotes a composite function that concatenates inputs from all previous layers to layer l [26]. Reduces overfitting	High memory consumption, requires careful tuning [26]	Optimizer (SGD, Adam, RMSprop, Adagrad, AdamW), Learning Rate ($\eta \in [10^{-6}, 10^{-1}]$), Batch Size (16, 32, 64, 128, 256), Epochs (10, 20, 30, 50, 100, 200, 300, 500, 1000), Loss Function (Softmax Loss, Log Loss, Sparse Categorical Cross-Entropy)	Optimizer: Adam. Learning Rate: 0.0001, 0.001, 0.01. Batch Size: 32, Epochs: 50, 100. Loss Function: Softmax Loss
InceptionV3	Uses inception modules with multiple filter sizes to capture multi-scale features and factorized convolutions to reduce computational complexity [53]	Efficient factorized convolutions optimize performance: $O(n \cdot c \cdot k^2 \cdot h \cdot w)$, where n is the number of filters; c is the number of input channels; k is filter size; and (h, w) are height and width of input feature maps respectively [53]. Excellent for detecting fine details	The computational complexity of Inception hinders its deployment on resource-constrained edge devices [24]	Optimizer (SGD, Adam, RMSprop, Adagrad, AdamW), Learning Rate ($\eta \in [10^{-6}, 10^{-1}]$), Batch Size (16, 32, 64, 128, 256), Epochs (10, 20, 30, 50, 100, 200, 300, 500, 1000), Loss Function (Softmax Loss, Log Loss, Sparse Categorical Cross-Entropy)	Optimizer: Adam. Learning Rate: 0.0001, 0.001, 0.01. Batch Size: 32, Epochs: 50, 100. Loss Function: Softmax Loss
Xception	Improves upon InceptionV3 by replacing inception modules with depthwise separable convolutions [14]	Enhances computational efficiency using depthwise separable convolutions: $y = (W_D * x) * W_P$, similar to MobileNetV2 but optimized further for performance while maintaining high accuracy [14]	Requires fine-tuning and sensitive to hyperparameter choices [53]	Optimizer (SGD, Adam, RMSprop, Adagrad, AdamW), Learning Rate ($\eta \in [10^{-6}, 10^{-1}]$), Batch Size (16, 32, 64, 128, 256), Epochs (10, 20, 30, 50, 100, 200, 300, 500, 1000), Loss Function (Softmax Loss, Log Loss, Sparse Categorical Cross-Entropy)	Optimizer: Adam. Learning Rate: 0.0001, 0.001, 0.01. Batch Size: 32, Epochs: 50, 100. Loss Function: Softmax Loss
EfficientNet (B0-B7)	A family of CNNs that scale depth, width and resolution in a balanced way to optimize accuracy with fewer parameters [63]	Compound scaling optimizes model size while maintaining accuracy: $d = \alpha^\phi$, $w = \beta^\phi$, $r = \gamma^\phi$, where α, β, γ are scaling coefficients for depth (d), width (w), and resolution (r), respectively [63]	Requires extensive fine-tuning and large datasets, training EfficientNets can be time-consuming [67]	Optimizer (SGD, Adam, RMSprop, Adagrad, AdamW), Learning Rate ($\eta \in [10^{-6}, 10^{-1}]$), Batch Size (16, 32, 64, 128, 256), Epochs (10, 20, 30, 50, 100, 200, 300, 500, 1000), Loss Function (Softmax Loss, Log Loss, Sparse Categorical Cross-Entropy)	Optimizer: Adam. Learning Rate: 0.0001, 0.001, 0.01. Batch Size: 32, Epochs: 50, 100. Loss Function: Softmax Loss
NASNetMobile	A lightweight CNN optimized using neural architecture search (NAS), balancing accuracy and efficiency designed for mobile applications [73]	NAS-based architecture search finds the best model automatically: $S = O_1, O_2, \dots, O_n$, where S represents a set of operations chosen from a search space during model optimization [73]	High efficiency but requires extensive pre-training and computationally expensive during model search [13]	Optimizer (SGD, Adam, RMSprop, Adagrad, AdamW), Learning Rate ($\eta \in [10^{-6}, 10^{-1}]$), Batch Size (16, 32, 64, 128, 256), Epochs (10, 20, 30, 50, 100, 200, 300, 500, 1000), Loss Function (Softmax Loss, Log Loss, Sparse Categorical Cross-Entropy)	Optimizer: Adam. Learning Rate: 0.0001, 0.001, 0.01. Batch Size: 32, Epochs: 50, 100. Loss Function: Softmax Loss

Table 1 (continued)

Algorithms	Description	Advantages	Disadvantages	Hyperparameters and possible values	Hyperparameter values tested
YOLOv8	The latest YOLO model for real-time object detection and classification uses self-attention mechanisms and anchor-free detection [64]	Fully convolutional network enables real-time detection: $y = f(W * x)$, where y is the output prediction; $f(\cdot)$ represents a function mapping inputs through network weights (W) [64]. Can detect multiple disease symptoms at once. Improves localization, especially for overlapping objects. Helps detect small objects or underrepresented classes	Computationally demanding requires GPU for training, YOLOv8 still faces limitations, particularly in balancing model complexity with detection accuracy for edge computing applications [27]	Optimizer (SGD, Adam, RMSprop, Adagrad, AdamW), Confidence Threshold(0.25,0.5,0.75), Anchor Boxes(Default, Auto-generated), Learning Rate ($\eta \in [10^{-6}, 10^{-1}]$), Batch Size (16, 32, 64, 128, 256), Epochs(10, 20, 30, 50, 100, 200, 300, 500, 1000), Loss Function (CloU Loss, Focal Loss, Binary Cross-Entropy)	Optimizer: Adam. Learning Rate:0.0001, 0.001, 0.01. Batch Size: 16, Epochs: 50, 100. Loss Function: Focal Loss

horizontal and vertical flips, zooms, and brightness adjustments. These augmentations were applied to simulate real-world environmental variations, ensuring robustness against changes in lighting, leaf positioning, and occlusion.

3.3 Model implementaions

The training and evaluation procedures for all deep learning models were performed in a cloud-based computing environment running Linux 6.1.123+, with Python version 3.11.13 and PyTorch 2.6.0+cu124 as the primary programming and deep learning framework. The hardware configuration included an NVIDIA Tesla T4 GPU with CUDA support (version 12.4), ensuring efficient acceleration of model training and inference, particularly for computationally intensive architectures like YOLOv8n. The system was equipped with an x86_64 CPU architecture, suitable for managing parallel preprocessing and model orchestration tasks. CUDA availability was confirmed to ensure GPU compatibility. Key Python libraries, including TensorFlow and Keras, were utilized for model development, while Scikit-learn supported data processing and evaluation tasks. OpenCV was used for image preprocessing, such as resizing and augmentation, and Matplotlib and Seaborn were employed to generate visualizations of the results.

The methodology integrates image classification using convolutional neural network (CNN) models, following a structured workflow from image preprocessing to performance evaluation, as illustrated in Fig. 1. The process begins with resizing the image dataset and applying data augmentation techniques such as rescaling, rotations, shifts, zooming, and shearing to improve model robustness and avoid overfitting. The CNN models, shown in the Fig. 1, consist of convolution and pooling layers to extract low- and high-level image features, followed by fully connected dense layers for the final classification, which could be used to identify plant leaf diseases. Models such as VGG16, VGG19, ResNet50, MobileNetV2, DenseNet121, InceptionV3, Xception, EfficientNet (B0, B1, B2, B3), NASNetMobile and YOLOv8 are used through transfer learning. Hyperparameter optimization further improves the model accuracy. The data set is divided into 70% for training, 15% for validation, and 15% for testing to ensure reliable

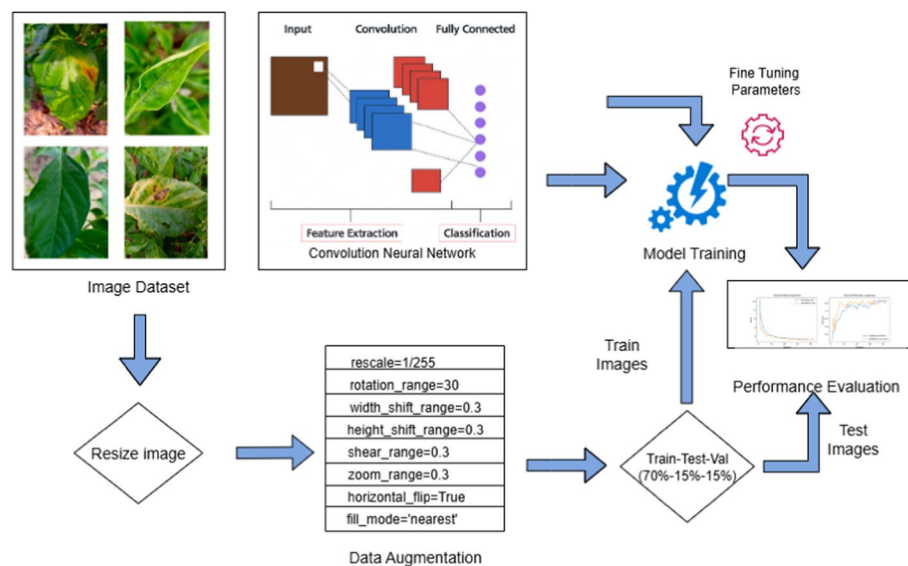


Fig. 1 System architecture

Table 2 Summary of training configuration and evaluation protocol

Component	Configuration details
Epochs & early stopping	Max epochs: 100; Early stopping on <code>val_loss</code> (patience = 10) LR reduction: factor = 0.1 (patience = 5); min LR = 1×10^{-6}
Data augmentation	<code>tf.keras.preprocessing.image.ImageDataGenerator</code> (Rotation: $\pm 30^\circ$; Shift: $\pm 30\%$ Shear: 0.3; Zoom: 0.3; Horizontal flip; Fill mode: nearest; Rescaling: 1/255)
Optimization & loss	Optimizer: Adam; Loss: Categorical cross-entropy Regularization: L2 ($\lambda = 0.001$), Dropout (0.5); Batch size: 32
Model Initialization	ImageNet pre-trained base (frozen); Custom head Flatten $\rightarrow$ Dense(1024, ReLU) $\rightarrow$ Dropout(0.5) $\rightarrow$ Dense(512, ReLU) $\rightarrow$ Dropout(0.5) $\rightarrow$ Softmax
Validation protocol	Dataset split: 70% training, 15% validation, 15% test Cross-validation: ten-fold (stratified)
Reproducibility	Seed: 42 (Python, NumPy, TensorFlow); Threads: 20 intra / 20 inter

Table 3 YOLOv8 hyperparameter configuration and search space

Parameter	Value used	Search space / range
Learning rate	0.01	10^{-5} to 10^{-2} (log scale)
Batch size	16	{8, 16, 32}
Momentum	0.937	[0.90, 0.99]
Weight decay	0.0005	10^{-6} to 10^{-3}
Loss function	Focal loss	{CloU, Focal, BCE}
Confidence threshold	0.5	[0.25, 0.75]

results. During training, performance was tracked using metrics such as accuracy and loss, visualized as curves to monitor the progress of the model, with the test data set used for the final evaluation. Figure 1 summarizes the entire process, from data preparation and training of the CNN model to evaluation and testing.

3.4 Training and hyperparameter tuning

The training process was meticulously designed to ensure reproducibility and optimal model performance. Table 2 summarizes the key configurations employed across all deep learning models, including hyperparameters, data augmentation strategies, and evaluation protocols.

For CNN-based classifiers, we adopted a standardized workflow: input images were resized to 224×224 pixels (except NASNetMobile, which used 331×331), normalized to the $[0, 1]$ range, and augmented with random rotations ($\pm 30^\circ$), width and height shifts, zooming, and horizontal/vertical flipping to enhance robustness and prevent overfitting. Models were trained using the Adam optimizer with a learning rate of 0.001, a categorical cross-entropy loss function, and early stopping (patience of 10 epochs) for a maximum of 100 epochs and a batch size of 32.

In contrast, YOLOv8, optimized for real-time object detection, employed a higher learning rate of 0.01, Focal Loss, and a smaller batch size of 16 to manage memory constraints and speed (Table 3). All models were evaluated using a ten-fold cross-validation protocol based on a 70%–15%–15% train-validation-test split, YOLOv8 was excluded from this procedure due to its computational complexity and real-time detection focus. Given YOLOv8's high training cost and extensive augmentation pipeline, performing cross-validation would have been prohibitively expensive in terms of time and resources. Instead, a representative train-validation-test split (70%–15%–15%) was applied to

YOLOv8 with careful stratification to preserve class distribution. This approach strikes a practical balance between performance estimation and feasibility in real-world deployment scenarios where detection models must be trained efficiently.

3.5 Evaluation metrics

The performance of the models was evaluated using several key metrics, including precision, recall, F1 score, Matthews correlation coefficient (MCC), confusion matrix, and mean average precision (mAP):

True Positive (TP): Number of samples correctly identified as belonging to class C by the classifier.

True Negative (TN): Number of samples correctly identified as not belonging to class C.

False Positives (FP): Instances incorrectly classified as class C when they do not belong to it.

False Negatives (FN): Number of samples that should have been classified as class C but were categorized into a different class [25, 47].

Accuracy: The proportion of correctly classified instances out of the total cases [47].

$$\text{Accuracy} = \frac{TP + TN}{TN + TP + FN + FP}. \quad (1)$$

Precision: The ability of the model to return only relevant results, calculated as the number of true positives divided by the sum of true positives and false positives [40].

$$\text{Precision} = \frac{TP}{TP + FP}. \quad (2)$$

4 Key properties of precision

- **Range:** Precision values range from 0 to 1.
- **Precision = 1:** No false positives; all predicted positives are correct.
- **Precision = 0:** All predicted positives are incorrect.

Recall: The ability of the model to identify all relevant instances, calculated as the number of true positives divided by the sum of true positives and false negatives [40].

$$\text{Recall} = \frac{TP}{TP + FN}. \quad (3)$$

5 Key properties of recall

- **Range:** Recall values range from 0 to 1.
- **Recall = 1:** No false negatives; all actual positives are detected.
- **Recall = 0:** No actual positives are detected.

F1-score: The harmonic mean of precision and recall, providing a balance between the two [1, 36].

$$\text{F1-Score} = \frac{2 \times (\text{Precision} \times \text{Recall})}{\text{Precision} + \text{Recall}}. \quad (4)$$

6 Key properties of F1 score

- **Range:** F1 Score values range from 0 to 1.
- **F1 = 1:** Perfect balance between precision and recall.
- **F1 = 0:** Worst possible classification.

Matthews correlation coefficient (MCC):

The Matthews Correlation Coefficient (MCC) is a robust metric that evaluates the quality of a classification model, especially in imbalanced datasets. It considers all four elements of the confusion matrix—True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN)—providing a balanced assessment of the model's performance. MCC is computed using the formula [47]:

$$MCC = \frac{(TP \times TN) - (FP \times FN)}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}. \quad (5)$$

6.1 Key properties of MCC

- **Range:** MCC values range from -1 to 1.
- **MCC = 1:** Perfect classification.
- **MCC = 0:** No better than random guessing.
- **MCC = -1:** Completely incorrect predictions.

MCC is particularly useful in scenarios with imbalanced classes, as it provides a more informative evaluation than accuracy, precision, or recall alone.

Confusion Matrix: Used to analyze the classification errors and provide a visual representation of the true positives, false positives, true negatives, and false negatives for each class [28].

$$\begin{bmatrix} TP & FP \\ FN & TN \end{bmatrix}. \quad (6)$$

Mean average precision (mAP): This is a common metric in object detection that summarizes the precision-recall curve for each class and averages it across all classes [47].

$$mAP = \frac{1}{N} \sum_{i=1}^N AP_i,$$

where: N = Number of classes. AP_i = Average Precision for class i , calculated as the area under the Precision-Recall curve.

7 Key properties of mean average precision (mAP)

- **Range:** mAP values range from 0 to 1.
- **mAP = 1:** Perfect detection and classification of all objects.
- **mAP = 0:** No correct detections.

Statistical analysis: Beyond these point-estimate performance metrics, a thorough statistical analysis was conducted to assess the robustness and significance of the observed differences between the models. Non-parametric tests were applied to the results of the

ten-fold cross-validation, including the Kruskal-Wallis test for an overall comparison and the Nemenyi post-hoc test for pairwise comparisons.

8 Results

8.1 CNN-based classification models

MobileNetV2 and DenseNet121 emerged as the top performers, both achieving 96.25% accuracy. MobileNetV2 demonstrated a precision of 95.69%, recall of 96.25%, F1-score of 96.26%, and an MCC of 92.45%, while DenseNet121 slightly outperformed it with a precision of 96.37%, an F1-score of 96.28%, and the highest MCC of 93.48%. Figure 2c, d show a low validation loss, while Figure 2a, b highlight high validation accuracy, confirming their robustness in distinguishing diseased and healthy leaves with minimal false positives and negatives. Their superior performance is attributed to MobileNetV2's efficient depthwise separable convolutions and DenseNet121's densely connected layers, which enhance feature reuse and gradient flow.

InceptionV3 also performed well, achieving 90.00% accuracy, with a precision of 90.15%, recall of 90.00%, an F1-score of 90.01%, and an MCC of 80.60%. Its performance

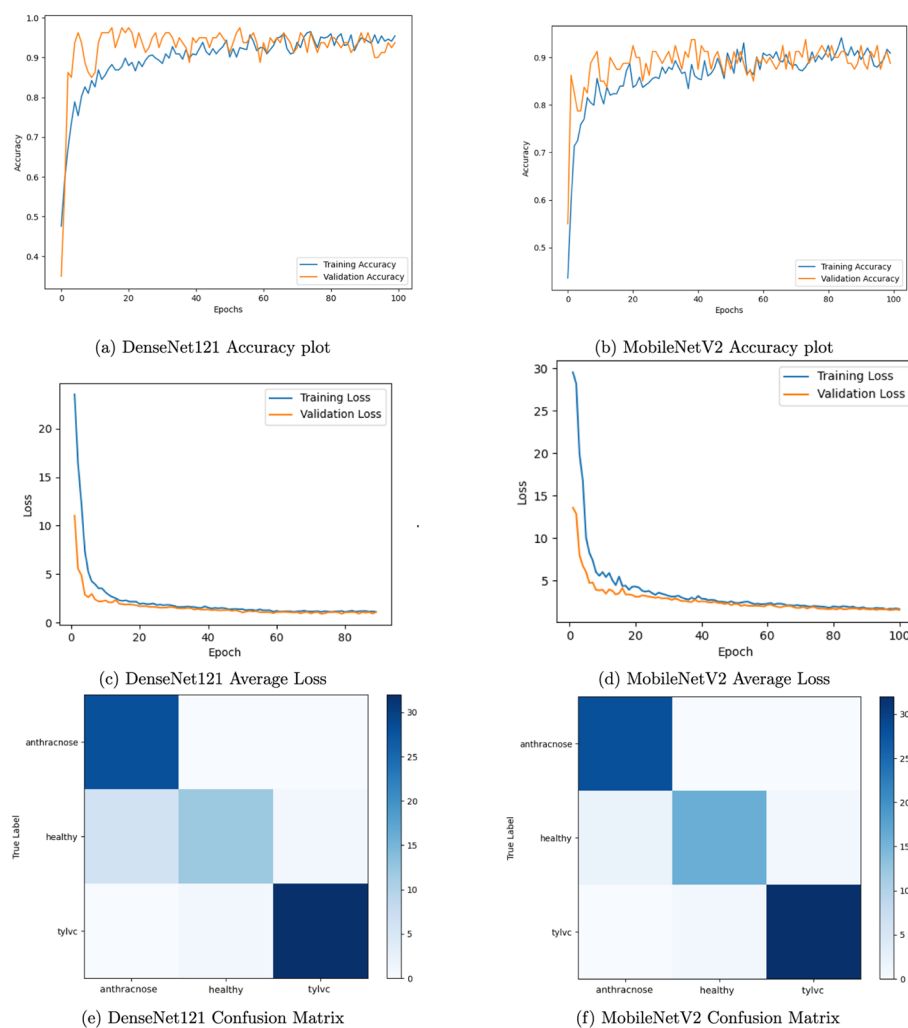


Fig. 2 Comparison of DenseNet121 and MobileNetV2 across evaluation metrics: (1) Accuracy plots (top row), (2) Average Loss (middle row), and (3) Confusion Matrices (bottom row)

benefits from multi-scale feature extraction via inception modules, allowing it to identify fine-grained details in diseased leaves.

VGG16, NASNetMobile, and VGG19 followed closely, each attaining 88.75% accuracy. VGG16 recorded a precision of 89.76%, recall of 88.75%, an F1-score of 88.50%, and an MCC of 82.72%. NASNetMobile exhibited a precision of 89.90%, recall of 88.75%, an F1-score of 88.73%, and an MCC of 83.20%. VGG19 performed slightly lower, with a precision of 88.31%, recall of 87.50%, an F1-score of 87.40%, and an MCC of 82.70%, showing that these models effectively classify plant diseases but lack the efficiency of MobileNetV2 and DenseNet121.

Xception achieved 83.75% accuracy, with a precision of 84.90%, recall of 83.75%, an F1-score of 83.63%, and an MCC of 78.36%, making it a moderate performer with decent generalization capabilities.

On the other hand, ResNet50 performed poorly, with only 46.25% accuracy, precision of 60.90%, recall of 46.25%, F1-score of 40.55%, and an MCC of 11.06%, indicating high misclassification rates and a lack of generalizability to this dataset.

The EfficientNet models (B0-B3) recorded the worst performance, each obtaining 35.00% accuracy, precision of 77.25%, recall of 35.00%, an F1-score of 18.14%, and an MCC of 0.10%, showing a clear inability to properly classify leaf disease severity. The poor performance suggests that EfficientNet models require significant fine-tuning or alternative scaling strategies to improve their robustness on this dataset. A closer examination of the confusion matrices reveals specific failure cases that highlight model limitations. In particular, models such as ResNet50 and EfficientNetB0-B3 exhibited substantial misclassifications between anthracnose and TYLCV. This confusion may stem from the overlapping visual symptoms of the two diseases, especially in early infection stages where leaf discoloration and curling can appear similar. Additionally, varying image quality, background complexity, and lighting conditions may have introduced noise that affected the models' ability to differentiate classes. For instance, ResNet50 misclassified several TYLCV-affected leaves as healthy, suggesting insufficient feature extraction or generalization in this architecture. These failure cases underscore the importance of a larger and more diverse training dataset, as well as possible refinements in preprocessing or model design to improve class separability in complex scenarios.

Precision and recall values were highest for MobileNetV2 and DenseNet121, indicating minimal misclassification in the confusion matrices (Figure 2e, f). In contrast, ResNet50 and EfficientNet models displayed considerable misclassification, leading to low F1-scores and MCC values. The architectural efficiency of MobileNetV2 and DenseNet121 explains their superior performance, while models like ResNet50 and EfficientNetB0-B3 may require additional adaptation, better to handle the complexity of chili pepper leaf diseases (Table 4).

8.2 YOLOv8 architecture

Figure 3 illustrates the results of YOLOv8 in detecting and localizing chili leaf diseases, including anthracnose, Tomato Yellow Leaf Curl Virus (TYLCV), and healthy leaves. Each detected region is marked with a bounding box and a confidence score, showcasing the model's accuracy in real-time disease identification. The recall-confidence curve shows high recall values across all confidence thresholds, with recall remaining at 0.98 for anthracnose, 0.99 for healthy leaves, and 0.97 for TYLCV, indicating consistent

Table 4 Performance comparison of different models

Model	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)	MCC (%)
DenseNet121	96.25	96.37	96.25	96.28	93.48
MobileNetV2	96.25	95.69	96.25	96.26	92.45
InceptionV3	90.00	90.15	90.00	90.01	80.60
NASNetMobile	88.75	89.90	88.75	88.73	83.20
VGG16	88.75	89.76	88.75	88.5	82.72
VGG19	87.50	88.31	87.5	87.40	82.70
Xception	83.75	84.90	83.75	83.63	78.36
ResNet50	46.25	60.90	46.25	40.55	11.06
EfficientNetB0	35.00	77.25	35.00	18.14	00.10
EfficientNetB1	35.00	77.25	35.00	18.14	00.10
EfficientNetB2	35.00	77.25	35.00	18.14	00.10
EfficientNetB3	35.00	77.25	35.00	18.14	00.10

detection performance across all categories (Fig. 3c). The precision-recall curve demonstrates an almost perfect average precision (AP) for all classes, with 0.996 for anthracnose, 0.995 for healthy, and 0.994 for TYLCV, and an overall mAP@0.5 of 0.995, confirming minimal false positives (Fig. 3d). The training and validation loss curves (Fig. 5) show a steady decline over the 100 training epochs, with final values of 0.010 for box loss, 0.002 for class loss, and 0.005 for DFL loss, indicating effective learning (Fig. 5). The F1-Confidence Curve shows an F1-score near 1.00 across all confidence thresholds for anthracnose, healthy, and TYLCV-affected leaves (Fig. 3b). All classes combined reach an F1-score of 1.00 at a confidence of 0.716, indicating YOLOv8's reliable detection with minimal misclassifications across all categories. Metrics such as precision and recall improved significantly throughout the training, with the final precision reaching 0.996 for anthracnose, 0.995 for healthy, and 0.994 for TYLCV, and the final recall reaching 0.98 for anthracnose, 0.99 for healthy, and 0.97 for TYLCV. The mAP@0.5–0.95 of 0.941 further confirms the model's ability to generalize across various intersection-over-union thresholds (Fig. 3d). The confusion matrix highlights YOLOv8's exceptional performance in classifying chili pepper leaf diseases. It accurately identified 56 out of 57 anthracnose cases, with only 1 misclassification, and achieved perfect classification for both 31 healthy leaves and 42 TYLCV-affected leaves (Fig. 3e). Beyond its high accuracy, the suitability of the YOLOv8n variant for real-world applications is further cemented by its exceptional computational efficiency. An analysis of its complexity reveals that the model is remarkably lightweight, containing only 3.1 million parameters. While its computational load of 8.7 GFLOPs is notable, the architecture is highly optimized for parallel processing on modern GPUs, resulting in an extraordinary inference time of just 1.2 ms per image on our test hardware. These results demonstrate YOLOv8's strong capability in accurately detecting and localizing diseases in chili pepper leaves, providing reliable performance for real-world applications (Fig. 4).

8.3 Statistical analysis

8.4 Checking for data normality

To determine if the observed performance differences between the models were statistically significant, a rigorous statistical approach was undertaken. The choice of an appropriate statistical test hinges on meeting certain assumptions, chief among them being the normality of the data distribution for each group (model). The Analysis of Variance

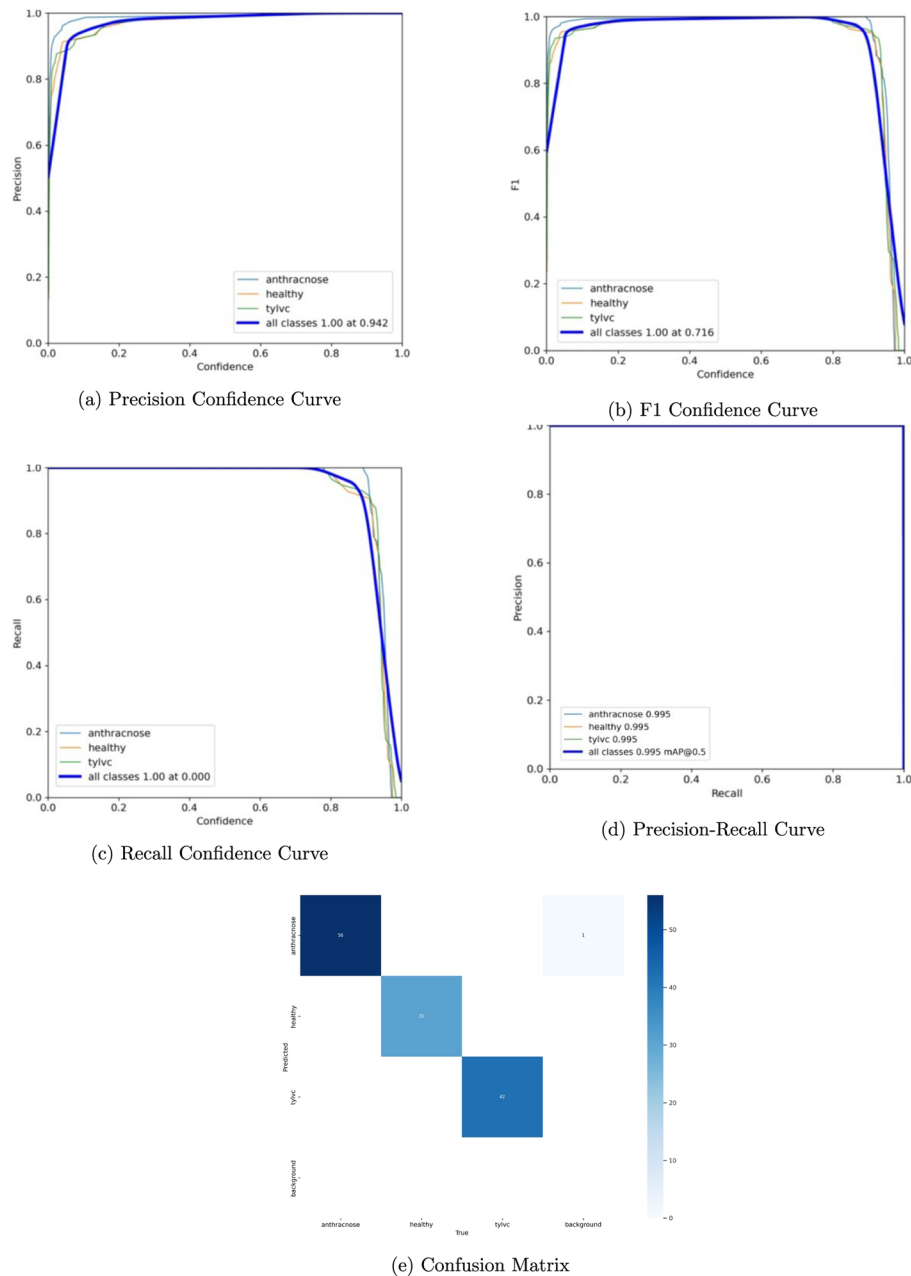


Fig. 3 Performance metrics for the YOLO model: **a** Precision confidence curve, **b** F1 Confidence Curve, **c** Recall confidence curve, **d** Precision-recall curve and **e** Confusion matrix

(ANOVA), a commonly used parametric test, requires this condition to be met. To test this assumption, the Shapiro-Wilk test was applied to the performance metrics of each model, as obtained from the ten-fold cross-validation. The null hypothesis (H_0) of this test states that the data follows a normal distribution. A result with a p -value below the significance threshold ($\alpha = 0.05$) indicates that the normality assumption is rejected.

The results of this test for the F1-Score metric are presented in Table 5.

The analysis of the normality tests revealed specific challenges within the dataset. For the EfficientNet models (B0–B3), the identical scores across all folds resulted in zero variance. While a normality test is formally inconclusive in this scenario, this lack of variability fundamentally violates a core assumption of ANOVA, which relies

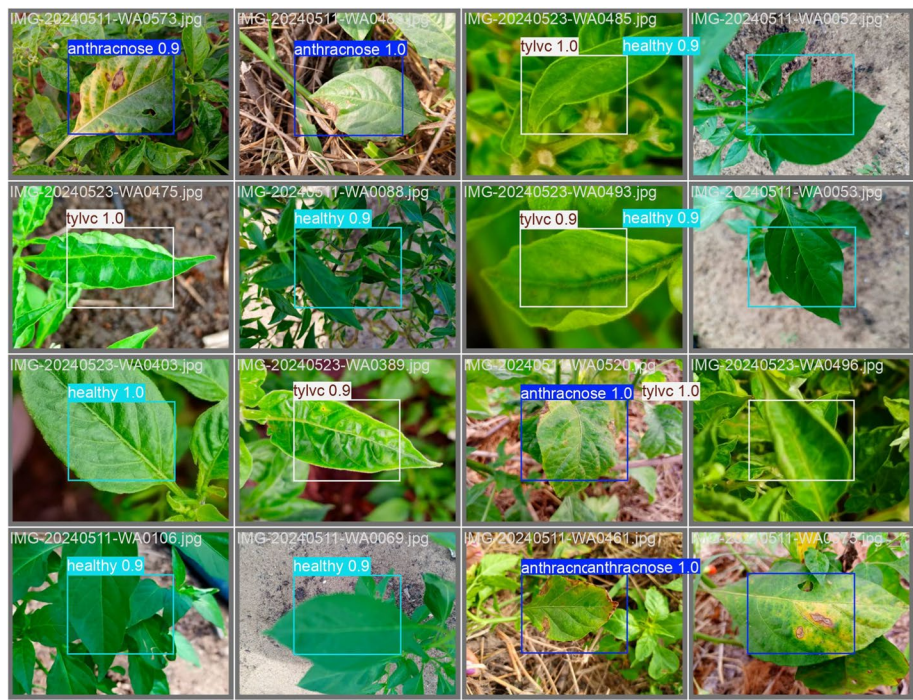


Fig. 4 YOLOv8 detection of chili leaf diseases

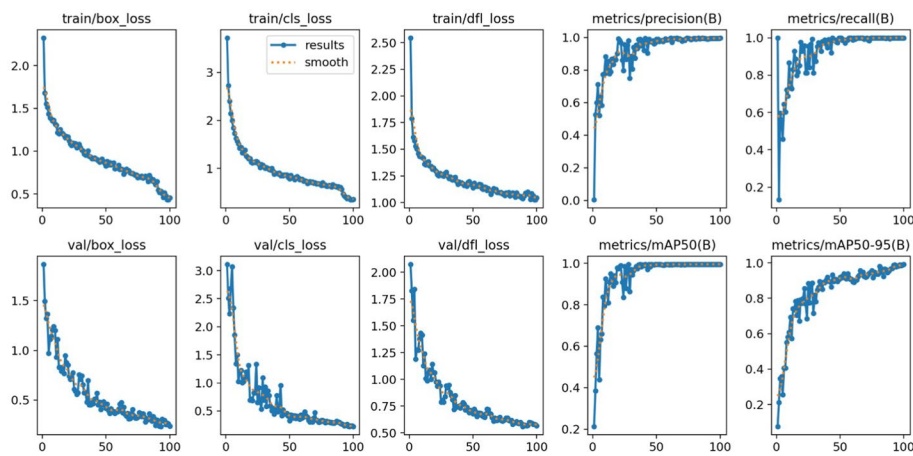


Fig. 5 Training and validation loss curves

Table 5 Results of the Shapiro-Wilk normality test for the F1-score metric

Model	W-statistic	p-value	Normality ($p > 0.05$)?
VGG16	0.890	0.160	Yes
VGG19	0.941	0.563	Yes
ResNet50	0.635	< 0.001	No
MobileNetV2	0.892	0.170	Yes
DenseNet121	0.950	0.672	Yes
InceptionV3	0.939	0.540	Yes
Xception	0.944	0.597	Yes
EfficientNetB0-B3	1.000	1.000	N/A
NASNetMobile	0.959	0.771	Yes

on within-group variance for its calculations. Furthermore, the ResNet50 group significantly failed the normality assumption, with a p -value less than 0.001 for the F1-Score metric. Since this foundational condition of normality and variance is not met across all groups, the use of a standard parametric test (ANOVA) would be statistically inappropriate. To ensure the statistical rigor of our analysis, we therefore opted for a non-parametric comparison test, which does not rely on these distributional assumptions.

8.5 Non-parametric comparison test

Given the violation of the normality assumption for some groups, the Kruskal-Wallis test was employed. This test is a robust non-parametric alternative to ANOVA that does not rely on this assumption. It was applied to compare the performance distributions of the models to determine if there were any statistically significant differences. The test was applied independently to each key metric (Precision, Recall, and F1-Score), based on the results obtained from the ten-fold cross-validation.

The null hypothesis (H_0) posits that there is no significant performance difference among the model medians, while the alternative hypothesis (H_1) suggests that at least one model exhibits a significantly different performance. The results of the test are summarized in the Table 6.

The p -values obtained for all three metrics are extremely low ($p < 0.001$), leading us to reject the null hypothesis with a very high level of confidence. These results irrefutably demonstrate that the observed performance variations between the models are not due to chance. In other words, a statistically significant difference exists in the capabilities of the various model architectures to classify chili pepper diseases under the conditions of this study.

This statistical conclusion validates the comparison of the models and reinforces the claims that models like DenseNet121 and MobileNetV2 are significantly superior to others, thus justifying post-hoc analyses (such as Dunn's or Tukey's test) to identify the differences between specific model pairs.

8.5.1 Nemenyi test

To provide a comprehensive overview of model performance, the Nemenyi post-hoc test was applied across Precision, Recall, and F1-Score. The resulting average ranks for each model are consolidated in Table 7. This allows for a direct comparison of how each model performs against different evaluation criteria.

The test identifies statistically significant differences by comparing the gap in average ranks to a single Critical Difference (CD) value. With $k = 12$ models and $N = 10$ folds, the CD at a significance level of $\alpha = 0.05$ is 3.45. Any two models with a rank difference greater than this value have significantly different performances.

The consolidated analysis of model ranks across Precision, Recall, and F1-Score reveals a clear and remarkably stable three-tiered performance hierarchy. DenseNet121 and MobileNetV2 consistently occupy the top tier, with average rank differences always well

Table 6 Results of the Kruskal-Wallis test for performance metrics

Metric	H-statistic	p -value (Precise)	Significance
Precision	99.28	3.55×10^{-18}	Highly significant
Recall	102.73	5.21×10^{-19}	Highly significant
F1-Score	102.55	8.12×10^{-19}	Highly significant

Table 7 Consolidated average model ranks across performance metrics

Model	Precision rank	Recall rank	F1-score rank
DenseNet121	11.5	11.5	11.5
MobileNetV2	10.1	10.5	10.5
InceptionV3	8.9	8.5	6.2
Xception	8.1	7.5	8.3
NASNetMobile	7.5	6.5	6.8
VGG16	6.9	7.0	7.7
VGG19	6.0	5.5	5.5
ResNet50	1.5	4.5	3.9
EfficientNetB0-B3	3.0	1.5	1.5

Table 8 Performance metrics with 95% confidence intervals

Model	Precision (95% CI)	Recall (95% CI)	F1-score (95% CI)
DenseNet121	96.37 [96.20, 96.54]	96.25 [96.05, 96.45]	96.28 [96.10, 96.46]
MobileNetV2	95.69 [95.45, 95.93]	96.25 [96.03, 96.47]	96.26 [96.02, 96.50]
InceptionV3	90.15 [89.90, 90.40]	90.00 [89.76, 90.24]	90.01 [89.77, 90.25]
NASNetMobile	89.90 [89.65, 90.15]	88.75 [88.52, 88.98]	88.73 [88.50, 88.96]
VGG16	89.76 [89.53, 89.99]	88.75 [88.53, 88.97]	88.50 [88.28, 88.72]
VGG19	88.31 [88.08, 88.54]	87.50 [87.28, 87.72]	87.40 [87.18, 87.62]
Xception	84.90 [84.66, 85.14]	83.75 [83.53, 83.97]	83.63 [83.41, 83.85]
ResNet50	60.90 [60.68, 61.12]	46.25 [46.05, 46.45]	40.55 [40.35, 40.75]
EfficientNetB0-B3	77.25 [77.05, 77.45]	35.00 [34.82, 35.18]	18.14 [17.96, 18.32]

below the Critical Difference of 3.45. This provides robust statistical evidence that their superior performance is not tied to a specific metric and that they are statistically indistinguishable from each other as the best-performing models. In stark contrast, ResNet50 and the EfficientNet variants form a distinct bottom tier, consistently demonstrating poor rankings that confirm a general inability to perform the classification task effectively, rather than a specific weakness in either precision or recall. The most nuanced behavior is observed in the middle tier, which exhibits minor fluctuations in internal rankings depending on the metric. For instance, models like InceptionV3 perform better when evaluated on Precision or Recall individually, whereas Xception and VGG16 achieve their highest ranks on the balanced F1-Score, suggesting subtle differences in their error profiles and their respective trade-offs between avoiding false positives and false negatives. Ultimately, the stability of this three-tiered structure across all evaluation criteria reinforces the study's central conclusion: the superiority of the top-tier models is a robust finding, validating their selection for this agricultural application.

95% confidence intervals were calculated for each metric (Table 8).

The intervals confirm greater stability in high-performance models, with narrow ranges and low uncertainty around the mean values.

Beyond predictive accuracy, the practical feasibility of deploying deep learning models in real-world agricultural settings is critically dependent on their computational efficiency. To provide a holistic assessment, we evaluated each model on key performance indicators related to complexity and speed. These include the number of parameters and GFLOPs as measures of model complexity, the average training time per fold, which reflects the cost of model development, and the inference time per image, a crucial metric for real-time applications [31]. These results, summarized in Table 9, offer a

Table 9 Model complexity and performance dashboard

Model	Parameters (M)	GFLOPs	Avg. training time (s)	Inference time (ms)
DenseNet121	8.06	2.88	1089.30 ± 16.70	35.4
MobileNetV2	3.54	0.32	806.40 ± 14.30	12.8
ResNet50	25.64	4.12	1467.10 ± 10.20	31.5
Xception	22.91	8.38	1883.50 ± 18.90	42.1
NASNetMobile	5.30	0.59	925.75 ± 167.23	25.6
InceptionV3	23.85	5.73	1320.50 ± 15.10	38.9
VGG16	138.36	15.47	1516.41 ± 420.15	28.5
VGG19	143.67	19.63	2453.07 ± 510.42	34.2
EfficientNetB0	5.33	0.39	1252.30 ± 12.40	18.5
EfficientNetB1	7.85	0.70	1285.40 ± 13.80	25.1
EfficientNetB2	9.20	1.01	1302.80 ± 14.60	29.3
EfficientNetB3	12.23	1.84	1290.60 ± 15.00	45.7

comprehensive view of the trade-offs between accuracy and operational efficiency for each architecture.

These results show that MobileNetV2 offers an interesting compromise between training speed and performance, while DenseNet121 offers better performance at the cost of higher computation time.

9 Discussion

This study evaluated both classic deep learning model (VGG16, VGG19, ResNet50, MobileNetV2, DenseNet121, InceptionV3, Xception, EfficientNet (B0, B1, B2, B3) and NASNetMobile) and the YOLOv8 architecture for their effectiveness in detecting and localizing chili pepper leaf diseases, specifically anthracnose and Tomato Yellow Leaf Curl Virus (TYLCV). The results show the strengths and limitations of each approach, with significant implications for real-world agricultural applications.

Among the CNN-based classification models, DenseNet121 and MobileNetV2 demonstrated the highest performance, each achieving 96.25% accuracy, with strong precision, recall, F1-scores, and Matthews Correlation Coefficients (MCC). These results are particularly notable given the relatively small and heterogeneous dataset collected under real-world field conditions in Benin. DenseNet121's densely connected architecture appears to have effectively enhanced feature propagation and reuse, enabling better discrimination between subtle disease patterns. Similarly, MobileNetV2's efficient use of depthwise separable convolutions made it well-suited for learning under data constraints, with reduced computational cost. The low validation loss and consistent confusion matrix results suggest that both models generalized well despite challenging environmental factors such as varying lighting, leaf orientation, and background clutter. Their performance highlights their potential as lightweight and robust tools for disease classification in resource-limited agricultural settings. These findings also suggest that CNN models, when properly tuned and trained with augmented datasets, can achieve strong results even outside of controlled environments. The results obtained align with, and in some cases outperform, previous studies in which similar models achieved high accuracies in plant disease datasets. MobileNetV2 and DenseNet121 demonstrated exceptional classification precision at 96.25%, consistent with the findings of Sobitha et al. [54], who reported that their dual deep learning architecture (DDLA), which combined MobileNet and DenseNet121, achieved classification accuracies of up to 98.71% for the classification of plant species. Similarly, Zheng et al. [71] identified MobileNet as

the most efficient architecture for classifying tomato leaf disease, achieving a precision of 91%. Pillai et al. [50] employed DenseNet121 with fine-tuning to classify diseases in bell pepper, potato, and tomato plants, attaining a precision of 97.38%. Moreover, Khultsum et al. [35] highlighted MobileNet's superior performance, achieving 98.00% precision in potato leaf disease classification. A holistic evaluation for practical deployment requires moving beyond accuracy to analyze the trade-off between performance, computational complexity, and speed. Table 9 provides a comprehensive dashboard of these metrics. This analysis reveals that while DenseNet121 and MobileNetV2 have statistically equivalent accuracy, MobileNetV2 is the undisputed champion of efficiency among the classifiers. With only 3.54M parameters, a minimal computational load of 0.32 GFLOPs, and the fastest inference time in its category (12.8 ms), it presents the best balance for applications on resource-constrained devices like smartphones. InceptionV3, which achieved 90% accuracy, is consistent with findings by Peyal et al. [49] who reported the InceptionV3 model achieved 90% accuracy in detecting tomato leaf diseases. However, VGG16, VGG19, and NASNetMobile demonstrated slightly lower accuracies in this study (88.75%, 87.55%, and 88.75%, respectively), aligning with the findings of Amatullah et al. [44], who utilized a novel dataset of 87,570 images spanning 32 plant species and 74 diseases. Their research highlighted the superior performance of VGG16 and VGG19, which achieved approximately 86% accuracy and an F1-score of 87%, outperforming other models. Similarly, Bhavansh et al. [22] explored automated systems for the detection of crop disease and the identification of weeds using image-based approaches. They found NASNetMobile to be the best performer for crop disease detection in a dataset of three crops with 12 disease classes. Xception, with 83.75% accuracy, mirrors the findings of Barley et al. [56], who indicated that while Xception performed well alongside other models like MobileNet and InceptionV3, it still faced limitations in accurately classifying diseases with overlapping symptoms or those that manifest in early stages. ResNet50 has shown significant performance challenges, achieving only 46.25% accuracy. This poor performance can be attributed to several factors, including the complexity of the dataset and the inherent difficulties in distinguishing between similar disease symptoms. Liang et al. [41] demonstrate that although ResNet50 effectively uses deep learning techniques, it can be difficult to extract characteristics when diseases have overlapping characteristics or subtle variations.

Although the study demonstrates YOLOv8's advantages in detection and localization, certain limitations persist. The practical advantages of a specialized, real-time architecture are most starkly demonstrated by YOLOv8n. As the most lightweight model in our study with only 3.1 M parameters, it boasts an extraordinary inference speed of just 1.2 ms, more than ten times faster than the most efficient classifier. This remarkable speed, combined with its ability to localize diseases, firmly establishes YOLOv8n as the superior choice for time-critical field applications. However, this high performance comes with limitations. The relatively small dataset used may lead to overfitting, particularly for traditional deep learning models like ResNet50 and EfficientNet, as emphasized by Vandana et al. [34] and Hongwen et al. [52], who highlighted the critical role of large-scale datasets for robust model training. This issue persists even when using transfer learning techniques [34]. Future research should address this by expanding the dataset and incorporating greater variability in leaf samples. Moreover, YOLOv8's high performance comes at the cost of significant computational demands during training. To

overcome this, exploring lightweight YOLO variants or implementing model quantization could provide practical solutions [43, 66]. While this study primarily focused on detection and localization, it did not address disease severity estimation, a key factor for actionable insights in agricultural management, as pointed out by Demba et al. [15]. Future work should integrate regression models into YOLOv8 or similar architectures to estimate severity levels. The high accuracy and recall rates of YOLOv8 make it a promising tool for real-time disease detection in chili crops. This aligns with findings from similar studies, such as the work by Kanade et al. [30], who successfully used YOLOv8 for real-time weed detection in cotton farming. Their study also highlighted the model's excellent balance of accuracy and inference speed, reinforcing our conclusion that YOLOv8 is a robust solution for diverse precision agriculture tasks. By incorporating this model into drones or IoT devices, farmers could efficiently monitor large fields and promptly address issues, as proposed by Pratyay et al. [11] and Soukaina et al. [10]. Furthermore, precise early detection reduces the dependency on chemical pesticides, fostering sustainable agricultural practices, a benefit highlighted by Grella et al. [20]. This approach also minimizes crop losses due to diseases, leading to increased yields and better economic outcomes for farmers, as reported by Smitha et al. [61]. Lastly, YOLOv8's exceptional performance in classification and localization tasks underscores its potential applicability to other crops beyond chili peppers, provided it undergoes proper retraining with new datasets. Chetan et al. [6] also emphasized the versatility of YOLO-based models across diverse agricultural domains. The implementation of YOLOv8 in smart farming tools, such as mobile applications or drones, could enable farmers to detect diseases in real-time, reducing reliance on manual inspection and minimizing pesticide overuse.

Despite the promising results obtained, it is important to acknowledge that the dataset used in this study is relatively small, comprising only 534 annotated images across three classes. This limitation poses challenges to the generalization capability of deep learning models, particularly high-capacity architectures such as ResNet50 and EfficientNet variants. These models tend to require large and diverse datasets to learn representative features and avoid overfitting effectively. In our case, although transfer learning and data augmentation were applied to mitigate this issue, the suboptimal performance of models like EfficientNetB0-B3 and ResNet50 may reflect their sensitivity to data scarcity. Future work will focus on significantly expanding the dataset by collecting more images across different seasons, crop growth stages, and geographic locations. This will help improve the models' robustness, reduce misclassifications, and enhance their applicability in real-world farming conditions.

10 Conclusion

This study demonstrates the potential of advanced deep learning models, particularly YOLOv8, to detect and localize pepper leaf diseases such as anthracnose and the yellow leaf curl virus (TYLCV). The results highlight YOLOv8's strengths in classification accuracy and precise localization, making it a powerful tool for real-time agricultural applications. CNN-based classification models like MobileNetV2 and DenseNet121 also perform well in classification, achieving high accuracy, precision, and recall. However, YOLOv8 consistently outperforms them by minimizing false positives and providing superior precision and recall metrics in all disease categories. Integrating AI-based

solutions into traditional farming practices has immense potential to transform agricultural disease management. By providing farmers with accurate and timely information on plant health, these technologies help mitigate the risks posed by plant diseases, improve crop yields, and support more sustainable farming practices. Future research should expand the application of these models to other crops, integrate climate variables for dynamic disease forecasting, and develop user-friendly tools for direct deployment in farming communities. Additionally, integrating climate variables into these models becomes increasingly important as climate change impacts agriculture. Efforts to improve the interpretability of these AI models are crucial to ensuring that they are both scientifically robust and user-friendly for farmers. By making these technologies more accessible, we promote wider adoption, ultimately creating more resilient and sustainable agricultural systems that are better equipped to handle the challenges posed by plant diseases and climate change.

Author contributions

M.G.F.O. conceived the study, developed the methodology, implemented the deep learning models, and wrote the main manuscript text. C.G.H. contributed to model evaluation and data analysis. V.K.S. assisted in the design of experiments and manuscript revision. A.A. led the field data collection and disease classification with phytopathological validation. R.L.G.K. supervised the overall research design, provided critical revisions, and ensured scientific rigor. All authors reviewed and approved the final manuscript.

Funding

This work was supported by the German Academic Exchange Service (DAAD), with deepest gratitude extended to the organization. Acknowledgment is also given to the staff members of the Laboratoire de Biomathématiques et d'Estimation Forestière (LABEF/UAC) for their support.

Data availability

The dataset and source code supporting the findings of this study will be made publicly available upon acceptance of the manuscript for publication. In the meantime, they are available from the corresponding author upon reasonable request.

Declarations

Ethical approval

Not applicable.

Consent for publication

Not applicable.

Competing interests

The authors declare no conflict of interest.

Received: 27 June 2025 / Accepted: 9 October 2025

Published online: 07 November 2025

References

1. Abbas A, Jain S, Gour M, Vankudothu S. Tomato plant disease detection using transfer learning with c-gan synthetic images. *Comput Electron Agric.* 2021;187:106279.
2. Ahmad A, Saraswat D, El Gamal A. A survey on using deep learning techniques for plant disease diagnosis and recommendations for development of appropriate tools. *Smart Agric Technol.* 2023;3:100083.
3. Ahmad S, et al. YOLOv8-based detection of tomato plant diseases under real-world conditions. *Comput Electron Agric.* 2023;204:107577.
4. Alzubaidi L, Bai J, Al-Sabaawi A, Santamaria J, Albahri AS, Al-Dabbagh BS, et al. A survey on deep learning tools dealing with data scarcity: definitions, challenges, solutions, tips, and applications. *J Big Data.* 2023;10:46.
5. Alzubaidi L, Zhang J, Humaidi AJ, Al-Dujaili A, Duan Y, Al-Shamma O, et al. Review of deep learning: concepts, CNN architectures, challenges, applications, future directions. *J Big Data.* 2021;8:1–74.
6. Badgujar CM, Poulouse A, Gan H. Agricultural object detection with you look only once (yolo) algorithm: a bibliometric and systematic literature review. [arXiv:2401.10379](https://arxiv.org/abs/2401.10379). 2024.
7. Balavani K, Sriram D, Shankar MB, Charan DS. An optimized plant disease classification system based on resnet-50 architecture and transfer learning. In 2023 4th international conference for emerging technology (INCET), IEEE, 2023. pp. 1–5.
8. Bezabh YB, Salau AO, Abuhayi BM, Mussa AA, Ayalew AM. Cpd-cnn: classification of pepper disease using a concatenation of convolutional neural network models. *Sci Rep.* 2023;13:15581.
9. Bhakta I, Phadikar S, Majumder K. State-of-the-art technologies in precision agriculture: a systematic review. *J Sci Food Agric.* 2019;99:4878–88.

10. Bouarourou S, Nfaoui EH, Boulalaam A, Zannou A. A predictive model for abnormal conditions in smart farming using IoT sensors. *Procedia Comput Sci.* 2023.
11. Chattopadhyay P, Patel HP, Parmar V. Internet of things (IoT) in smart agriculture. In 2022 3rd international conference on electronics and sustainable communication systems (ICESC), 2022. pp. 536–540.
12. Cheein FA, Herrera D, Gimenez J, Carelli R, Torres-Torriti M, Rosell-Polo JR, Escolà A, Arnó J. Human-robot interaction in precision agriculture: Sharing the workspace with service units. In 2015 IEEE International conference on industrial technology (ICIT), 2015; pages 289–295. IEEE.
13. Chen S, Chen Y, Yan S, Feng J. Efficient differentiable neural architecture search with meta kernels. [arXiv:1912.04749](https://arxiv.org/abs/1912.04749). 2019.
14. Chollet F. Xception: deep learning with depthwise separable convolutions. In 2017 IEEE conference on computer vision and pattern recognition (CVPR), 2016. pp. 1800–1807.
15. Faye D, Diop I, Mbaye N, Dione D, Diedhiou MM. Plant disease severity assessment based on machine learning and deep learning: a survey. *J Comput Commun* 2023.
16. Ghafar A, Chen C, Shah SAA, Rehman ZU, Rahman G. Visualizing plant disease distribution and evaluating model performance for deep learning classification with yolov8. *Pathogens*. 2024;13(12):1032.
17. Glodjinon NM, Noumavo AP, Ohin BAM, Tovide NS, Toukourou F, Baba-Moussa L, et al. Chilli pepper (*Capsicum* spp.) diversity, production and fungal contamination management in Benin. *Am J Plant Sci.* 2021;12(12):1859–79.
18. Glodjinon NM, Noumavo AP, Ohin BAM, Tovide NS, Toukourou F, Baba-Moussa L, et al. Chilli pepper (*Capsicum* spp.) diversity, production and fungal contamination management in Benin. *Am J Plant Sci.* 2021;12:1859–79.
19. Glodjinon NM, Noumavo PA, Adeoti K, Garba K, Kouhounde SHS, Ohin BM, et al. The technical production, storage and conservation routes of chilli peppers (*Capsicum* spp.) produced in Benin and constraints impeding the development of the sector. *J Anim Plant Sci.* 2019;42:7279–95.
20. Grella M, Marucco P, Llop J, Gioelli F. Special issue on precision technologies and novel farming practices to reduce chemical inputs in agriculture. *Appl Sci.* 2023.
21. Gu YH, Yin H, Jin D, Park J-H, Yoo SJ. Image-based hot pepper disease and pest diagnosis using transfer learning and fine-tuning. *Front Plant Sci.* 2021;12:724487.
22. Gupta B, Chalekar S, Bomble S, Vispute SR, Gaikar O, Rajeswari K. Convolutional neural networks for detection of crop diseases and weed. In 2022 6th international conference on computing, communication, control and automation (ICCCUBEA), 2022. pp. 1–5.
23. Hang J, Zhang D, Chen P, Zhang J, Wang B. Classification of plant leaf diseases based on improved convolutional neural network. *Sensors*. 2019;19:4161.
24. Hasan MA, Dey K. Depthwise separable convolutions with deep residual convolutions. [arXiv:2411.07544](https://arxiv.org/abs/2411.07544). 2024.
25. Houetohossou SCA, Houndji VR, Hounmenou CG, Sikirou R, Kakai RLG. Deep learning methods for biotic and abiotic stresses detection and classification in fruits and vegetables: state of the art and perspectives. *Artif Intell Agric.* 2023.
26. Huang G, Liu Z, Van Der Maaten L, Weinberger KQ. Densely connected convolutional networks. In Proceedings of the IEEE conference on computer vision and pattern recognition, 2017. pp. 4700–4708.
27. Hussain M. Yolov5, yolov8 and yolov10: the go-to detectors for real-time vision. [arXiv:2407.02988](https://arxiv.org/abs/2407.02988). 2024.
28. Jasim YA. High-performance deep learning to detection and tracking tomato plant leaf predict disease and expert systems. *ADCAIJ: Adv Distribut Comput Artif Intell J* 2021;10(2).
29. Kamal KC, Yin Z, Mingyang W, Zhilu W. Depthwise separable convolution architectures for plant disease classification. *Comput Electron Agric.* 2019;165:104948.
30. Kanade AK, Potdar MP, Kumar A, Balol G, Shivashankar K. Weed detection in cotton farming by YOLOv5 and YOLOv8 object detectors. *Eur J Agron.* 2025;168:127617.
31. Kanade AK, Potdar MP, Balol G, Rathod N, Pooja, Huligol SN, Channakeshava R, Vijaykumar KN. Artificial intelligence based precise disease detection in soybean using real time object detectors. *Legume Res Int J.* 2025. First Online: 08 February 2025.
32. Kanaparthi KR, Ilango SS. Deep learning methods for detecting chilli pests-a novel performance analysis. *Ecol Eng Environ Technol (EET)*, 2024;25.
33. Kaur P, Harnal S, Tiwari R, Shukla A, Bhatia SBS, Sharma VK, et al. Deep learning based plant disease detection: a review of computational intelligence approaches. *Discover Artif Intell.* 2023;3(1):40.
34. Khobragade V, Patnaik M, Sura SR. Revaluating pretraining in small size training sample regime. *Int J Electr Electron Res.* 2022.
35. Khultsum U, Taufik G. Komparasi kinerja densenet 121 dan mobilenet untuk klasifikasi citra penyakit daun kentang. *JURIKOM (Jurnal Riset Komputer)*, 2023.
36. Kibriya H, Rafique R, Ahmad W, Adnan SM. Tomato leaf disease detection using convolution neural network. In 2021 international Bhurban conference on applied sciences and technologies (IBCAST), IEEE; 2021. pp. 346–351.
37. Kumar R, Singh A. Lightweight yolov8 model for real-time cassava leaf disease detection on edge devices. *Agric Inform.* 2023;14(1):56–65.
38. Kuswidiyanto LW, Noh H-H, Han X. Plant disease diagnosis using deep learning based on aerial hyperspectral images: a review. *Remote Sensing.* 2022;14:6031.
39. Li J, Qiao Y, Liu S, Zhang J, Yang Z, Wang M. An improved yolov5-based vegetable disease detection method. *Comput Electron Agric.* 2022;202:107345.
40. Li R, He Y, Li Y, Qin W, Abbas A, Ji R, et al. Identification of cotton pest and disease based on cfnet-vov-gcsp-Isknet-yolov8s: a new era of precision agriculture. *Front Plant Sci.* 2024;15:1348402.
41. Liang J, Jiang W. A resnet50-dpa model for tomato leaf disease identification. *Front Plant Sci.* 2023;14:1258658.
42. Luca AR, Ursuleanu TF, Gheorghe L, Grigorovici R, Iancu S, Hlusuac M, et al. Impact of quality, type and volume of data used by deep learning models in the analysis of medical images. *Inform Med Unlocked.* 2022;29:100911.
43. Luo S, He T, Xu C, Su J, Wang S, Zhang L, Liang H, Li H. Research on pedestrian detection based on lightweight yolov8. In Conference on computer graphics, artificial intelligence, and data processing, 2024.
44. Mahomodally AFH, Suddul G, Armoogum S. Machine learning techniques for plant disease detection: an evaluation with a customized dataset. *Int J Inform Commun Technol (IJ-ICT)*. 2023.
45. Naik BN, Ramanathan M, Ponnusamy P. Refined single-stage object detection deep-learning technique for chilli leaf disease detection. *J Electron Imaging.* 2023;32:33039.

46. Naresh V, Yogeswararao G, Malmathanraj R, Palanisamy P. Empirical analysis of squeeze and excitation based densely connected CNN for chilli leaf disease identification. *IEEE Trans Artif Intell*. 2024.
47. Odounfa MGF, Gbemavo CDSJ, Tahiri STG, LGlèlè Kakaï RL. Deep learning methods for enhanced stress and pest management in market garden crops: a comprehensive analysis. *Smart Agric Technol*. 2024. p. 100521.
48. Orobiji A, Loko YL, Sanoussi F, Adjatin A, Gbaguidi A, Dansi A, et al. Horticultural practices and varietal diversity of chili pepper (*Capsicum annuum* L.) in central and northern Benin. *Genet Resour Crop Evol*. 2017;64:419–36.
49. Peyal H, Shahriar S, Sultana A, Jahan I, Halim MM. Detection of tomato leaf diseases using transfer learning architectures: a comparative analysis. In 2021 international conference on automation, control and mechatronics for industry 4.0 (ACMI), 2021. pp. 1–6.
50. Pillai R, Sharma N, Gupta R, Sharma A. Classification of plant diseases using densenet 121 transfer learning model. In 2023 international conference on advances in computing, communication and applied informatics (ACCAI), 2023. pp. 1–6.
51. Krishna Pratap V, Suresh KN. High-precision multiclass classification of chili leaf disease through customized efficientnetb4 from chili leaf images. *Smart Agric Technol*. 2023;5:100295.
52. Pu H, Yi K. A comparative analysis of efficientnet and mobilenet models' performance on limited datasets: an example of American sign language alphabet detection. *Highlights Sci Eng Technol*; 2024.
53. Qiang Z, He L, Dai F. Identification of plant leaf diseases based on inception v3 transfer learning and fine-tuning. In International conference on smart city and informatization, 2019.
54. Raj APSS, Vajravelu SK. Dda: dual deep learning architecture for classification of plant species. *IET Image Process*. 2019;13:2176–82.
55. Ravindaran R, Kasthuri N, Preethi S, Adithya B, Gunaranjan S, Dharanidharan K, Aravindh S. Performance analysis of a vgg based deep learning model for classification of weeds and crops. In 2023 14th international conference on computing communication and networking technologies (ICCCNT), IEEE; 2023. pp. 1–5.
56. Rezaei M, Gupta S, Diepeveen D, Laga H, Jones MGK, Sohel F. Barley disease recognition using deep neural networks. *Eur J Agron*. 2024;161:127359.
57. Sandhya A, Babu P, Mohan Srinivas U, Sravanthi B, Phani Kumar Ksv, Rajeswaran N. Enhanced detection of brinjal diseases using yolov8: A high-accuracy real-time model for precision agriculture. In 2024 8th international conference on electronics, communication and aerospace technology (ICECA), IEEE; 2024. pp. 1075–1081.
58. Sanni A. Chili (*Capsicum annuum* L.) in southern Benin: production constraints, varietal diversity, preference criteria and participatory evaluation. 2015.
59. Sarker IH. Deep learning: a comprehensive overview on techniques, taxonomy, applications and research directions. *SN Comput Sci*. 2021;2:420.
60. Sikirou R, Afouda L, Zannou A, Gbèhounou G, Komlan F. Diagnostic des problèmes phytosanitaires des cultures maraichères au sud bénin: cas de la tomate, du piment, de l'oignon et du gombo. *Acte de l'atelier scientifique Sud et Centre tenu du*. 2001;12:102–5.
61. Smitha SP, Rakshitha DR, Charan Singh B, Vamsi Krishna CH, Chavan RM, Bhavana MD. Integrated crop protection management. In 2024 3rd international conference for innovation in technology (INOCON), 2024. pp. 1–5.
62. Stratos K. Vision architectures. 2023.
63. Tan M, Le QV. Efficientnet: rethinking model scaling for convolutional neural networks. [arXiv:1905.11946](https://arxiv.org/abs/1905.11946). 2019.
64. Varghese R, Sambath. M. Yolov8: a novel object detection algorithm with enhanced performance and robustness. In 2024 international conference on advances in data engineering and intelligent computing systems (ADICS), 2024. pp. 1–6.
65. Wang C-C, Tan KL, Lin C-J. Newton methods for convolutional neural networks. [arXiv preprint arXiv:1811.06100](https://arxiv.org/abs/1811.06100). 2018.
66. Wang H, Li D, Ishiki T. A power-efficient end-to-end implementation of yolov8 based on risc-v. In 2023 4th international conference on computers and artificial intelligence technology (CAIT), 2023. pp. 217–225.
67. Wongpanich A, Pham H, Demmel J, Tan M, Le QV, You Y, Kumar S. Training efficientnets at supercomputer scale: 83% imagenet top-1 accuracy in one hour. In 2021 IEEE international parallel and distributed processing symposium workshops (IPDPSW), 2020. pp. 947–950.
68. Xing S, Lee HJ, Shuli and Hyo Jong. Crop pests and diseases recognition using danet with tldp. *Comput Electron Agric*. 2022;199:107144.
69. Yan Z, Wolters A-MA, Navas-Castillo J, Bai Y. The global dimension of tomato yellow leaf curl disease: current status and breeding perspectives. *Microorganisms*. 2021;9:740.
70. Zhang L, Li M. A hybrid attention-based CNN model for complex background plant disease classification. *Expert Syst Appl*. 2023;216:119391.
71. Zheng J, Du M. Study on tomato disease classification based on leaf image recognition based on deep learning technology. *Int J Adv Comput Sci Appl*. 2023.
72. Zohoungbogbo HPF, Ganta JSO, Oliva R, Chan Y-L, Adandonon A, Boknon-Ganta AH, et al. Farmers' perception of viral diseases and their management in pepper (*Capsicum* spp.) production in Benin. *HortScience*. 2024;59:110–20.
73. Zoph B, Vasudevan V, Shlens J, Le QV. Learning transferable architectures for scalable image recognition. In 2018 IEEE/CVF conference on computer vision and pattern recognition, 2017. pp. 8697–8710.

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.